

Crazyflie + Vicon Hover — Replication Guide (cflib_python, USC-CATT stack)

End-to-end setup for flying a Crazyflie 2.1 under Vicon motion capture using the **USC-CATT** firmware and the **cflib_python** off-board controllers.

Goes from a bare machine + unflashed drone to a working hover test. A Python controller on the laptop streams **raw PWM** to the 4 motors — this is why the USC-CATT firmware fork is required, and it's the natural place to experiment with custom control laws.

Paths / usernames: Everything is rooted at your **home folder** (`~`). Commands use `~` so they work regardless of username. The Vicon SDK unzips into a build-hash subfolder (e.g. `20230413_145507h`) that will differ for your download — adjust it where noted.

Tested step-by-step on **Ubuntu 22.04 / Linux**.

0. Repository layout (target)

After all clones you'll have:

```
~/crazyflie-firmware/           # USC-CATT fork (flashed to the drone)
~/ros2_ws/
├── src/
│   ├── sala4_crazyflie/
│   └── cflib_python/           # controllers, missions, motorRawTest.py
~/vicon-datastream-sdk-1.12/    # Vicon DataStream SDK 1.12
```

1. Install cfclient + cflib (Bitcraze) and Crazyradio permissions

Use the **latest official** Bitcraze instructions:

- cflib (Python lib): <https://www.bitcraze.io/documentation/repository/crazyflie-lib-python/master/installation/install/>
- cfclient (GUI): <https://www.bitcraze.io/documentation/repository/crazyflie-clients-python/master/installation/install/>

Typical install:

```
sudo apt update
sudo apt install -y git python3-pip
pip3 install cflib
pip3 install cfclient
```

Crazyradio USB permissions (udev) — required so the dongle is usable without root.

Follow the Bitcraze "USB permissions" guide:

[https://www.bitcraze.io/documentation/repository/crazyflie-lib-](https://www.bitcraze.io/documentation/repository/crazyflie-lib-python/master/installation/usb_permissions/)

[python/master/installation/usb_permissions/](https://www.bitcraze.io/documentation/repository/crazyflie-lib-python/master/installation/usb_permissions/) This installs the udev rules and adds you to the `plugdev` group. Log out/in (or replug the dongle) afterwards. Verify:

```
groups | grep plugdev      # you should be in plugdev
lsusb | grep -i bitcraze   # dongle detected
```

2. Clone + build the USC-CATT firmware

```
cd ~
git clone https://github.com/USC-CATT/crazyflie-firmware.git
cd ~/crazyflie-firmware
git submodule update --init --recursive      # REQUIRED: vendor/CMSIS must not be e
```

If `vendor/CMSIS` is empty you'll hit a missing `arm_math.h` at build time — that means submodules weren't initialized. Re-run the line above.

Build dependencies + firmware:

```
sudo apt install -y swig make gcc-arm-none-eabi build-essential
cd ~/crazyflie-firmware
make cf2_defconfig      # config for the Crazyflie 2.1 board
make -j$(nproc)         # produces build/cf2.bin
```

Optional, only if scripting against the firmware in Python: `make bindings_python`.

Why this fork: USC-CATT adds the raw motor-command CRTP path

(`crtp_motor_command_raw.c` , `motorRawInit`) that the `cflib_python` controllers push raw PWM into. Stock Bitcraze firmware will **not** work with this flight path.

3. Flash the drone (cfclient GUI)

1. Put the drone in **bootloader mode**: hold the power button ~3 s until the blue LEDs blink.

2. Plug in the Crazyradio dongle.
3. Launch the client and flash:

```
cfclient
```

In the GUI: **Connect** ▶ **Bootloader** tab → **Initiate bootloader cold boot** → browse to `~/crazyflie-firmware/build/cf2.bin` → **Program**.

4. Wait for it to finish writing all pages, then restart the drone normally.

CLI alternative (flash over radio without the GUI):

```
cd ~/crazyflie-firmware
make cload # drone in bootloader mode, dongle plugged in
```

Success = it writes all ~284 pages of `cf2.bin` to completion.

4. Clone sala4_crazyflie + raw motor sanity test

```
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws/src
git clone https://github.com/USC-CATT/sala4_crazyflie.git
cd ~/ros2_ws/src/sala4_crazyflie/cflib_python
```

Find your drone's URI — it is drone-specific. Scan for it and use *your* value everywhere a URI appears below:

```
python3 -c "import cflib.crtp; cflib.crtp.init_drivers(); print(cflib.crtp.scan_in
```

Example URI (this lab's drone): `radio://0/100/2M/E7E7E7E701` . Yours may differ — do not assume a channel/address; use what the scan reports.

Raw motor test (confirms radio link + all 4 motors respond, before Vicon):

```
python3 motorRawTest.py
```

⚠ Props off or drone secured — this spins motors. If the URI in the script differs from your scan result, set it to your value first.

5. Install the Vicon DataStream SDK 1.12

Full manual guide: see `Vicon_DataStream_SDK_1.12_Ubuntu22_Manual_Install.md` .
Summary (adjust the build-hash subfolder to match your download):

```
# Download DataStream SDK 1.12 (Linux x64) from:
# https://www.vicon.com/downloads/utilities-and-sdk/datastream-sdk
# Save the .zip to ~/Downloads, then:
sudo apt install -y unzip build-essential
mkdir -p ~/vicon-datastream-sdk-1.12
unzip -o ~/Downloads/ViconDataStreamSDK_1.12_*.zip -d ~/vicon-datastream-sdk-1.12

# Note the extracted folder (build hash varies), e.g.:
SDK_LIB=~/.vicon-datastream-sdk-1.12/20230413_145507h/Release/Linux64
ls "$SDK_LIB"      # expect DataStreamClient.h and libViconDataStreamSDK_CPP.so
```

Make the runtime library findable (persist in `~/.bashrc`):

```
echo 'export LD_LIBRARY_PATH="\"$SDK_LIB\":${LD_LIBRARY_PATH}\"' >> ~/.bashrc
source ~/.bashrc
echo "$LD_LIBRARY_PATH"
```

6. Network + Vicon connectivity

6a. Second Ethernet interface → Vicon network

The Vicon PC is `192.168.10.1` . Configure this computer's **second Ethernet** NIC (the cable to the Vicon switch) with a static address:

- IPv4 address: `192.168.10.2`
- Netmask: `255.255.255.0` (/24)
- Gateway: `192.168.10.1`

GUI: NetworkManager ▸ Wired ▸ IPv4 ▸ Manual. Or CLI:

```
nmcli con up "Wired connection 1"      # if the profile exists but isn't active
ping 192.168.10.1                       # must succeed before continuing
```

No DHCP on this subnet — the static config above is required.

6b. Vicon GUI setup (Nexus / Tracker / Shogun)

On the Vicon PC, in its GUI:

1. **Mask** the cameras (mask out reflections/static bright spots).
2. Calibrate if needed.
3. Place the drone, create an **object/subject**, and **name it**. Note this name — it must match exactly later (this lab uses `Crazy_Test`).

6c. Smoke test + live tracking test

```
cd ~/vicon-datastream-sdk-1.12
./smoke_test          # prints SDK version + a Connect() result code
./live_tracking_test  # enumerates live subject/segment names – confirms streami
```

`smoke_test` returning a non-zero Connect code just means no server was reachable at `127.0.0.1:801` ; point it at `192.168.10.1` / run it against the live server to confirm real data. `live_tracking_test` should list your subject (e.g. `Crazy_Test`).

7. Fly — hover test

Terminology: "the bridge" below is the custom program `live_tracking_bridge` (in `cflib_python/vicon_bridge_cpp/`) that reads Vicon over the network and forwards pose to the Python controller over local UDP. It is separate from the network link in Section 6.

Terminal 1 — start the bridge (uses your subject name + the Vicon IP):

```
cd ~/ros2_ws/src/sala4_crazyflie/cflib_python/vicon_bridge_cpp
./live_tracking_bridge 192.168.10.1 Crazy_Test 51001
```

Streams `x y z qx qy qz qw` (metres) over UDP → `127.0.0.1:51001` → `vicon_bridge.py` . Healthy = `[Vicon] ~200 fps | ... | errors 0` . If it connects but streams **0 packets**, the subject name doesn't match the Vicon scene → fix it in Section 6b.

Terminal 2 — run the controller in your **own focused terminal** (the spacebar killswitch uses an in-process `pynput` listener that only fires in a focused window, never a background/headless shell):

```
cd ~/ros2_ws/src/sala4_crazyflie/cflib_python
python3 pid_pwm_position_viconpos.py
```

- `pid_pwm_position_viconpos.py` (`send_extpos` , **position only** + onboard EKF for attitude) = **preferred**, robust to Vicon orientation flips.

- `pid_pwm_position_viconpose.py` (`send_extpose` , position + Vicon orientation) = only if the subject orientation is stable / axis-aligned.
- Set hover target in-file: `HOVER_HEIGHT` + `USER_DEFINED_TRAJECTORY` .
- Auto-lands to 0.05 m after the last waypoint unless you pass `--no-land` .
- Logs position vs setpoint to `waypoint_data.json` .
- URI: set it in the script to **your** scanned value (Section 4).

8. Adding your own controller

Each controller is a standalone Python script in `cflib_python/` — no reflashing needed, iterate in seconds.

1. Copy the closest template:

```
cd ~/ros2_ws/src/sala4_crazyflie/cflib_python
cp pid_pwm_position_viconpos.py my_controller.py
```

2. Keep the plumbing: Vicon UDP intake (`vicon_bridge.py`), `send_extpos` + logging, the spacebar killswitch, and the raw-motor output call.
3. Replace the **control law** in the loop — compute thrust/torques → convert to the 4 raw PWM values → send. Tune gains, `HOVER_HEIGHT` , trajectory in-file.
4. Run: `python3 my_controller.py` .

Existing examples to learn from: `augmented_pid_pwm_position.py` , `mellinger_pwm_position.py` .

Safety

- **SPACEBAR** = kill all motors immediately. Keep the controller terminal focused for the entire flight.
- Keys **1 / 2 / 3 / 4** = trim individual motor to 0.65x.
- Props off / drone secured for `motorRawTest.py` and first bring-up.

Reference links

- USC-CATT org: <https://github.com/USC-CATT>
- Firmware: <https://github.com/USC-CATT/crazyflie-firmware>
- sala4_crazyflie: https://github.com/USC-CATT/sala4_crazyflie

- CIIIP install: <https://www.bitcraze.io/documentation/repository/crazyflie-iiip-python/master/installation/install/>
- cfclient install: <https://www.bitcraze.io/documentation/repository/crazyflie-clients-python/master/installation/install/>
- Vicon DataStream SDK: <https://www.vicon.com/software/datastream-sdk/>
- Related work (reference): <https://github.com/OmAcharya-avtr/crazyflie-missions> , <https://github.com/OmAcharya-avtr/crazyflie-iiip-workspace>